Learning Objectives

By the end of this session, you should be able to:

- Understand the difference between left-associative and right-associative operators and explain why assignments are right-associative.
- Extend an expression grammar to support assignments with the correct precedence and associativity.
- Adapt a recursive-descent parser to handle right-associative assignments alongside left-associative arithmetic operators.
- Distinguish between *lvalues* and *rvalues* when generating code for assignments.
- Compile and execute programs that combine arithmetic expressions, assignments, and variable declarations.
- Explain why live registers must be preserved across function calls and implement caller-side saving and restoring on the stack.

Step 1 — A Preview of Assignment Code Generation

Before extending our compiler with assignment expressions, let's review what we already implemented in Sessions 19 and 20. Consider the following program:

```
global x;

fn main()
{
    x = 42;
    return x;
}
```

Our compiler cannot parse this program yet. However, the code generator already provides everything needed to translate it into assembly. Create `xtest_gen_04.abc`:

```
@ "gen.hdr"
@ "symtab.hdr"

fn main()
{
    genInit();

    // global x: i64;
    genAddGlobal("x");

    // fn main() {
    genFuncDefBegin("main");

    // x = 42;
    local rVal: GenReg = genConstant("42");
    local lValAddr: GenReg = genLoadAddr("x");
    genStore(rVal, lValAddr);
    genReleaseReg(rVal);
    genReleaseReg(lValAddr);

    // return x;
    local xVal: GenReg = genLoadVar("x");
    genReturn(xVal);
    genReleaseReg(xVal);
    genFuncDefEnd();
}
```

Compile and run the test program. The generated program should return `42`.

Notice how the assignment `x = 42` is translated:

- `genConstant("42")` computes the value of the right-hand side.
- `genLoadAddr("x")` obtains the address of the left-hand side.
- `genStore(rVal, lValAddr)` stores the value at that address.

This illustrates an important distinction: **The left-hand side of an assignment specifies where to store a value, while the right-hand side computes the value to be stored.**

In this worksheet, we will teach our parser and expression trees to represent this distinction so that the compiler can generate such code automatically.

© Michael C. Lehn | CC BY-SA 4.0

Step 2 — Adding Assignment Expressions

In Step 1, we saw that our code generator already supports everything needed to assign a value to a variable. Now we want to represent assignments in our expression trees. Eventually, our compiler should be able to translate an expression such as `x = 42` into the corresponding assembly instructions.

Extending the Expression Tree

In `expr.hdr`, add `EXPR_ASSIGN` to the binary operator group of `ExprKind`. In `expr.abc`, extend `printExprTree()` with the following case at the appropriate position:

```
case EXPR_ASSIGN: printf("{=} "); break;
```

Why do we print `{=}` rather than simply `=`? The answer has to do with how we visualize expression trees using LuaLaTeX.

A Question of Left and Right

An assignment differs from ordinary arithmetic expressions. For example, in `x = 42` the right-hand side computes a value, while the left-hand side identifies a location where that value should be stored.

The left-hand side must therefore be an **lvalue** (*left value*): an expression that designates a storage location. For now, our compiler supports only variable identifiers as lvalues. Later, other expressions may qualify as well.

Add the following helper function to `expr.abc`:

```
fn isLValueExpr(expr: ->Expr): bool
{
    if (expr->kind == EXPR_IDENTIFIER) {
        return true;
    } else {
        return false;
    }
}
```

Declare it in `expr.hdr`:

```
extern fn isLValueExpr(expr: ->Expr): bool;
```

Our parser will eventually use this function to reject invalid assignments such as

```
42 = x;
```

The parser already checks whether identifiers used in expressions have been declared. For assignments, it must additionally check whether the left-hand side is an lvalue.

Load the Address of an Expression

We also need a helper function to obtain the address of an lvalue. While `loadExpr()` computes the value of an expression, `loadExprAddr()` computes the address of the storage location it represents.

For now, only identifier expressions are supported. Add the following function to `expr.abc`:

```
fn loadExprAddr(expr: ->Expr): GenReg
{
    if (expr->kind == EXPR_IDENTIFIER) {
        return genLoadAddr(expr->primary.identifier);
    } else {
        assert(0 && "not an lvalue");
        return 0;
    }
}
```

© Michael C. Lehn | CC BY-SA 4.0

Generating Code for Assignments

Extend the binary expression handling in `loadExpr()` with a special case for `EXPR_ASSIGN`:

```
if (expr->kind == EXPR_ASSIGN) {
    assert(isLValueExpr(expr->binary.left));

    local leftAddr: GenReg = loadExprAddr(expr->binary.left);
    local rightVal: GenReg = loadExpr(expr->binary.right);

    genStore(rightVal, leftAddr);
    genReleaseReg(leftAddr);

    return rightVal;
} else {
    // ... as before
}
```

Why do we release `leftAddr` but not `rightVal`? Because an assignment is itself an expression, and its value is the value that was assigned. This will become important when we implement expression statements such as

```
a = b = 42;
```

As usual, the caller of `loadExpr()` is responsible for releasing the returned register.

Testing the Expression Tree

Create `xtest_expr_assign.abc`:

```
@ <stdio.hdr>
@ "gen.hdr"
@ "expr.hdr"

fn main()
{
    local expr: ->Expr;
    assert(genAddGlobal(UStrCreate("x")));

    expr = createBinaryExpr(EXPR_ASSIGN,
                            createIdentifierExpr(UStrCreate("x")),
                            createIntegerExpr(UStrCreate("42")));

    printf("\t# load expression: ");
    printExprTree(expr);
    printf("\n");

    genReleaseReg(loadExpr(expr));

    releaseExpr(expr);
}
```

Compile and run the test. Inspect the generated assembly and verify that it stores the value `42` in the global variable `x`.

The generated assembly is not yet a complete executable program. That is fine for now: we are testing the expression code generator in isolation. Soon, our compiler will allow us to test assignments directly in complete programs.

Step 3 — Adding Assignments to the Grammar

In Session 13, we started with a recursive-descent parser for arithmetic expressions. Later, we introduced `tokenKindPrec()` and `parseBinary()` to make it easy to support additional binary operators without introducing a separate parsing function for every precedence level.
Our expression grammar has since grown to support more operators, function calls, and variable references. Its basic structure, however, is still:

```
expr                 = binary
binary               = binary-additive
binary-additive      = binary-multiplicative { ("+" | "-") binary-multiplicative }
binary-multiplicative = factor { ("*" | "/" | "%") factor }
factor               = decimal-literal
                     | character-literal
                     | identifier [ "(" [ expr { "," expr } ] ")" ]
                     | "(" expr ")"
```

© Michael C. Lehn | CC BY-SA 4.0

The rules `binary`, `binary-additive`, and `binary-multiplicative` are all handled by `parseBinary()`. The function `tokenKindPrec()` determines which operators belong to which precedence level. We now want to support assignment expressions in statements such as

```
x = 42;
```

and even

```
a = b = 42;
```

The natural approach is to extend our grammar in the same way we introduced the other binary operators. Since `=` has lower precedence than `+` and `-`, we introduce a new grammar rule above binary:

```
expr       = assignment
assignment = binary { "=" binary }
```

This follows the same pattern as our existing grammar rules for additive and multiplicative operators. The assignment operator has the lowest precedence, so the new rule becomes the starting point for parsing expressions.
In principle, we could simply implement this new rule by extending functions `tokenKindPrec()` and `exprKindFromToken()` to recognize `=` as another binary operator. After all, making it easy to add binary operators was precisely why we introduced these functions in Session 13.
This time, however, we will deliberately take a different approach!
Instead of modifying `parseBinary()`, we will implement the new grammar rule using a separate function, `parseAssignment()`.
This will also give us an opportunity to examine more closely how the structure of a grammar affects the expression trees produced by our parser.

Tasks

1. **Implement the new grammar rule.** In `parseexpr.abc`, add the forward declaration

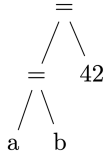
   ```
   fn parseAssignment(): ->Expr;
   ```

   Then modify `parseExpr()` and implement `parseAssignment()` as follows:

   ```
   fn parseExpr(): ->Expr
   {
       return parseAssignment();
   }

   fn parseAssignment(): ->Expr
   {
       local left: ->Expr = parseBinary(1);
       if (!left) {
           return nullptr;
       }
       while (token.kind == EQUAL) {
           getToken();
           local right: ->Expr = parseBinary(1);
           if (!right) {
               error("expression expected");
           }
           left = createBinaryExpr(EXPR_ASSIGN, left, right);
       }
       return left;
   }
   ```

   Use `make tree.tex` to visualize the expression tree generated by the parser for `a = b = 42`. You should see the following:

   It represents the expression `(a = b) = 42`.

   This is called **left associativity**: operators of the same precedence are grouped from left to right. But there is a problem. Recall that the left-hand side of an assignment must be an *lvalue*.

© Michael C. Lehn | CC BY-SA 4.0

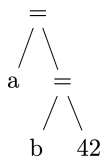
In our compiler, only identifier expressions qualify as lvalues. Look at the tree. What is the left-hand side of the outer assignment? Why can't our compiler generate code for it?

2. **Fix the grammar and parser.** We actually want the tree corresponding to `a = (b = 42)`. This is called right associativity: operators of the same precedence are grouped from right to left.

   Modify our grammar rule accordingly:

   ```
   assignment = binary [ "=" assignment ]
   ```

   Notice that we have replaced the repetition `{ "=" binary }` with an optional recursive assignment. Now adapt `parseAssignment()` yourself to implement this grammar. Rebuild the compiler and generate the expression tree for `a = b = 42` again. Verify that it now has the intended structure:

   Finally, explain to your neighbor why the new grammar produces a right-associative expression tree.

3. **Checking for lvalues.** In Step 2, we introduced `isLValueExpr()` to determine whether an expression can appear on the left-hand side of an assignment. However, our parser does not use this function yet. Let's fix that! In `parseAssignment()`, insert the following code at the appropriate position[1]:

   ```
   if (!isLValueExpr(left)) {
       error("operator can only be applied to an lvalue");
   }
   ```

   Rebuild the compiler and test both valid and invalid assignments. In particular, try:

   ```
   a = b = 42;
   42 = a;
   (a + b) = 42;
   ```

   The first assignment should be accepted, while the other two should produce an error. Why is it important to check the left-hand side during parsing rather than waiting until code generation?

Putting It All Together — and Looking Ahead

We can now parse assignment expressions, check that their left-hand sides are lvalues, and generate code for them. Time to put everything together!

Create `examples/ws21_assign.not-abc`:

```
fn foo(a, b)
{
    a = b;
    return a;
}

fn main()
{
    return foo(237, 42);
}
```

Compile and execute the program:

```
./not-abc < examples/ws21_assign.not-abc > foo.s
ulmas foo.s -o foo
ulm foo
echo $?
```

The expected exit code is `42`.

Our compiler can now handle assignments! However, there is still something missing: **declarations of global and local variables**. Without them, our test programs are rather limited.

---

[1] The right place is immediately after recognizing the `=` operator, but before calling `getToken()`. ;-)

© Michael C. Lehn | CC BY-SA 4.0

Let's make the example more interesting:

```
global x, y;

fn foo(a, b)
{
    x = a;
    y = b;

    local A, B;
    A = y;
    B = x;

    return A - B;
}

fn main()
{
    return foo(42, 84);
}
```

This program stores the function arguments in global variables, copies them into local variables, and finally returns their difference.

Try compiling it. Our compiler cannot handle it yet because it does not support global and local variable declarations in the source language.

But that is exactly what we will implement in the next step!

Step 4 — Global and Local Variable Declarations

Our code generator already supports global and local variables. In Session 20, we implemented `genAddGlobal()` and `genAddLocal()` to register variables in the symbol table and reserve storage for them.

What is still missing is the ability to declare these variables in our source language.

We want to distinguish two kinds of declarations:

- **Global variables** are declared outside functions, at the top level of a program.
- **Local variables** are declared inside functions, as part of a compound statement.

Let's extend our grammar accordingly:

```
top-level = { global | fn } EOI
global = "global" identifier { "," identifier } ";"
local = "local" identifier { "," identifier } ";"
fn = "fn" identifier "(" [ identifier { "," identifier } ] ")" compound-stmt
compound-stmt = "{" { expr-stmt | return-stmt | local } "}"
expr-stmt = expr ";"
```

Notice that global declarations are allowed only at the top level, while local declarations are allowed only inside compound statements.

Unlike the changes to expression parsing in the previous step, these changes concern the structure of entire programs. We will therefore modify parser.abc, not `parseexpr.abc`.

Tasks

1. Add forward declarations. In `parser.abc`, add:

   ```
   fn parseGlobal(): bool;
   fn parseLocal(): bool;
   ```

2. **Extend the top-level parser.** Modify `parseTopLevel()` to recognize global declarations as well as function definitions:

   ```
   fn parseTopLevel(): bool
   {
       getToken();
       while (parseGlobal() || parseFn()) {
       }
       return token.kind == EOI;
   }
   ```

   Notice how closely the while loop follows the new grammar rule for `top-level`.

© Michael C. Lehn | CC BY-SA 4.0

3. **Implement global declarations.** Add the following function to `parser.abc`:

```
fn parseGlobal(): bool
{
    if (token.kind != GLOBAL) {
        return false;
    }
    getToken();
    while (token.kind == IDENTIFIER) {
        genAddGlobal(token.val) || error("already defined");
        getToken();
        if (token.kind != COMMA) {
            break;
        }
        getToken();
    }
    expected(SEMICOLON);
    getToken();
    return true;
}
```

   Each identifier is passed to `genAddGlobal()`, which registers it in the global symbol table. If the name has already been defined, we report an error.
4. **Implement local declarations.** Implement `parseLocal()` analogously to `parseGlobal()`, using `genAddLocal()` instead of `genAddGlobal()`.
5. **Extend compound statements.** Modify `parseCompoundStmt()` to recognize local variable declarations as well.
6. **Build and test.** Rebuild the compiler and compile `examples/ws21_assign.not-abc`. Assemble and execute the generated program. The expected exit code is `42`.

Step 5 — A Small Hole in Our Parser

Try adding the following declaration to your test program:

```
global ;
```

Surprisingly, our compiler accepts it without reporting an error! But according to our grammar, a global declaration must contain at least one identifier:

```
global = "global" identifier { "," identifier } ";"
```

The same applies to local declarations.

Tasks:

- Find the problem in `parseGlobal()` and `parseLocal()`.
- Fix both functions by inserting `expected(IDENTIFIER);` at the appropriate places.

Make sure your parser also rejects declarations such as:

```
global x, ;
local a, ;
```

Finally, verify that valid declarations still work.

Step 6 — What's wrong?

Our compiler can now handle assignments to local variables. But consider the following program, `examples/ws21_registers.not-abc`:

```
fn main()
{
    local ch;
    ch = getchar();
    putchar(ch);
    return ch;
}
```

Compile and execute it. Does it behave as expected? Use the Debugger!

© Michael C. Lehn | CC BY-SA 4.0

Step 7 — Function Calls: Saving and Restoring Registers

When generating the assignment `ch = getchar()`, our compiler first loads the address of `ch` into a register, say `%4`. It then generates the call to `getchar()`. But `getchar()` also uses registers! When the function returns, `%4` may no longer contain the address of `ch`. Consequently, the assignment stores the return value at the wrong address.

In practice, calling conventions specify which registers must be preserved by the caller and which by the callee. For our compiler, we use a simple rule: **The callee may overwrite all registers starting at `%4`.** The caller must therefore save all registers currently in use before a function call and restore them afterward.

Saving and Restoring Registers

Add the following global variables and helper functions to `gen.abc`:

```
global savedReg: array[16] of int;
global numSavedReg: int;

fn saveRegs()
{
    numSavedReg = 0;
    if (!numUsedReg) {
        return;
    }
    setSegment(TEXT);
    printf("\t# start: save used registers on stack\n");
    for (local r: GenReg = 0; r < sizeof(usedReg)/sizeof(usedReg[0]); ++r) {
        savedReg[r] = usedReg[r];
        if (savedReg[r]) {
            ++numSavedReg;
            printf("\tmovq \t%%%d, \t-8 * %d(%%1)\n", r, numSavedReg);
        }
    }
    printf("\tsubq \t8 * %d, \t%%1, \t%%1\n", numSavedReg);
    printf("\t# end: save used registers on stack\n");
}

fn restoreRegs()
{
    if (!numSavedReg) {
        return;
    }
    setSegment(TEXT);
    printf("\t# start: restore used registers on stack\n");
    printf("\taddq \t8 * %d, \t%%1, \t%%1\n", numSavedReg);
    local numRestoredRegs: int = 0;
    for (local r: GenReg = 0; r < sizeof(usedReg)/sizeof(usedReg[0]); ++r) {
        if (savedReg[r]) {
            ++numRestoredRegs;
            printf("\tmovq \t-8 * %d(%%1), \t%%%d\n", numRestoredRegs, r);
            savedReg[r] = 0;
        }
    }
    assert(numRestoredRegs == numSavedReg);
    numSavedReg = 0;
    printf("\t# end: restore used registers on stack\n");
}
```

Next, modify `genCallBegin()` to call `saveRegs()` before preparing the function call, and modify `genCallEnd()` to call `restoreRegs()` after retrieving the return value and restoring the stack pointer.

Tasks

1. **Test the fix.** Rebuild the compiler and execute `examples/ws21_registers.not-abc` again. Verify that the character is echoed and returned correctly.
2. **Draw the stack.** Sketch the stack immediately before calling `getchar()` and immediately after returning from it, assuming `%4` contains the address of `ch`. Indicate where `%4` is saved, where the function's return value is stored, and how the stack pointer changes.
3. **Explain it to your neighbor.** Why do we need the separate array `savedReg`? Why can't `restoreRegs()` simply inspect `usedReg`? **Hint:** Between saving and restoring the registers, `genCallEnd()` allocates another register for the return value.

© Michael C. Lehn | CC BY-SA 4.0